

# 50-microservices-hands-on

## □ Microservices Hands-On



Figure 1: Microservices Architecture

**Level:** Advanced

**Prerequisites:** Modul 6 (Node.js Express), Modul 21 (Docker)

**Output:** 3 microservices + API Gateway

---

## □ Deskripsi

Modul ini memberikan pengalaman hands-on membangun arsitektur microservices dari nol. Mulai dari memecah monolith menjadi

service-service terpisah, mengelola komunikasi antar service (REST, gRPC, message queue), hingga menyatukan semuanya di belakang API Gateway.

Peserta akan membangun 3 microservices (User Service, Product Service, Order Service) yang saling berkomunikasi, masing-masing dengan database sendiri, plus sebuah API Gateway sebagai entry point terpusat.

---

## □ Tujuan Pembelajaran

- Memahami prinsip bounded context dan pemecahan monolith
- Mampu mengelola multi-service dengan Docker Compose
- Mengimplementasikan komunikasi REST, gRPC, dan message queue
- Membangun API Gateway dengan routing, auth, dan rate limiting

---

## □ Sesi

| Sesi | Judul                       | Durasi    |
|------|-----------------------------|-----------|
| 01   | Monolith to Microservices   | 120 menit |
| 02   | Docker Multi-Service        | 120 menit |
| 03   | Inter-Service Communication | 120 menit |
| 04   | API Gateway                 | 120 menit |

**Total durasi:** 480 menit (8 jam)

---

## □ Tech Stack

| Komponen          | Teknologi                                           |
|-------------------|-----------------------------------------------------|
| Runtime           | Node.js + TypeScript                                |
| Container         | Docker + Docker Compose                             |
| Database          | PostgreSQL (User), MongoDB (Product), MySQL (Order) |
| Message Queue     | RabbitMQ                                            |
| API Gateway       | Traefik                                             |
| Service Discovery | DNS / Consul                                        |
| Monitoring        | Centralized logging (winston + ELK)                 |

---

## □ Struktur Repository

```
50-microservices-hands-on/  
├── README.md  
├── 01-monolith-to-micro.md  
├── 02-docker-multi-service.md  
├── 03-inter-service-comm.md  
├── 04-api-gateway.md  
└── projects/  
    ├── user-service/  
    ├── product-service/  
    ├── order-service/  
    └── gateway/
```

---

## □ Output Akhir

1. **User Service** — auth, profil, manajemen user (PostgreSQL)
2. **Product Service** — katalog, stok, harga (MongoDB)
3. **Order Service** — checkout, riwayat pesanan (MySQL)
4. **API Gateway** — routing, rate limiting, auth gateway (Traefik)

Semua service berjalan di container Docker, saling berkomunikasi via REST internal + message queue, dengan API Gateway sebagai single entry point.

## Sesi 01: Monolith to Microservices

**Durasi:** 120 menit

---

## □ Tujuan

- Memahami masalah arsitektur monolith
  - Menerapkan Domain-Driven Design untuk bounded context
  - Menggunakan Strangler Fig pattern untuk migrasi
  - Merancang service boundaries yang tepat
-

## 1. Masalah Arsitektur Monolith

### 1.1 Deployment Coupling

Dalam monolith, satu perubahan kecil membutuhkan deploy seluruh aplikasi. Contoh nyata:

```
// monolith/src/order.ts – satu perubahan di order  
function calculateShipping(order: Order) {  
  // fix: tax calculation for international orders  
}
```

```
// deploy seluruh app – termasuk user, product, payment  
// padahal cuma berubah satu fungsi
```

**Akibat:** - Deploy risk tinggi — satu error bisa bikin semua fitur down  
- Release cycle lambat — harus tunggu fitur lain selesai - Rollback susah — harus rollback semuanya

### 1.2 Scaling Terbatas

```
// monolith – semua fitur dalam satu proses  
const app = express();  
app.use('/api/users', userRoutes);    // heavy I/O  
app.use('/api/products', productRoutes); // read-heavy  
app.use('/api/orders', orderRoutes);   // write-heavy  
app.use('/api/payments', paymentRoutes); // compliance-heavy  
  
// Mau scale cuma bagian payment? Ga bisa.  
// Harus scale seluruh app – boros resource.  
app.listen(3000);
```

**Masalah:** Semua fitur scale bareng. Bagian yang berat (payment) butuh resource besar, bagian ringan (products) ikut dibawa.

### 1.3 Technology Lock-In

Monolith biasanya pilih satu tech stack. Susah ganti: - Mau coba database baru? Ga bisa — seluruh app tergantung database lama  
- Mau pake bahasa lain? Ga bisa — semua dalam 1 codebase - Mau upgrade library? Resiko tinggi — bisa break fitur lain

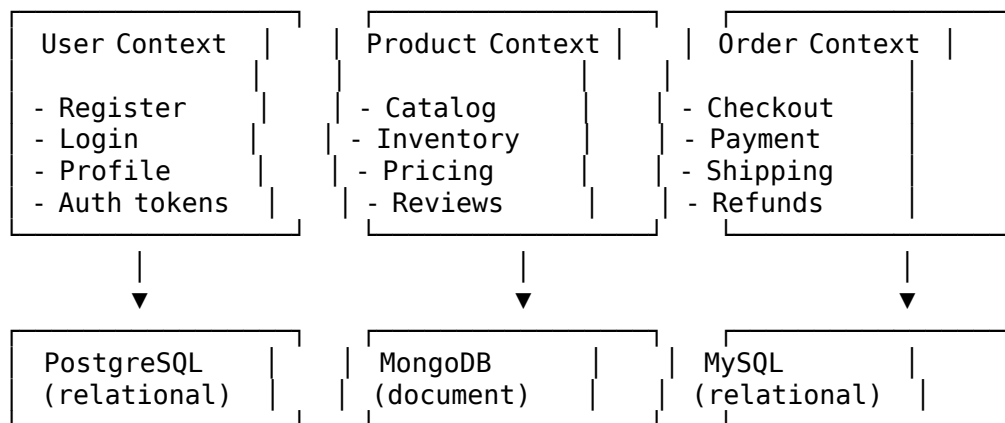
## 2. Bounded Context — Domain-Driven Design

### 2.1 Apa Itu Bounded Context?

Bounded context adalah batasan logis dalam domain bisnis. Setiap context punya: - **Model data sendiri** — tabel/collection terpisah - **Bahasa sendiri** — Ubiquitous Language - **Tim sendiri** — ownership jelas

### 2.2 Identifikasi Bounded Context

Ambil contoh e-commerce:



**Setiap context punya database sendiri.** Ini kunci microservices.

### 2.3 Ubiquitous Language

Setiap context punya istilah sendiri. Contoh:

| Istilah | User Context | Product Context | Order Context |
|---------|--------------|-----------------|---------------|
| User    | Customer     | Seller          | Buyer         |
| Product | -            | Item, SKU       | Merchandise   |
| Order   | Transaction  | -               | Purchase      |
| Stock   | -            | Inventory       | Availability  |

### 2.4 Mapping Domain

Langkah identifikasi:

```
// Step 1: Identifikasi domain events
const domainEvents = {
```

```

    'UserRegistered',    // -> User Context
    'ProductAdded',      // -> Product Context
    'ProductSold',       // -> Order Context
    'PaymentProcessed',  // -> Order Context
    'OrderShipped',      // -> Order Context
    'ReviewSubmitted',   // -> Product Context
  ];

  // Step 2: Kelompokkan per context
  const boundedContexts = {
    user: {
      events: ['UserRegistered'],
      entities: ['User', 'Role', 'Permission'],
      services: ['AuthService', 'ProfileService'],
    },
    product: {
      events: ['ProductAdded', 'ReviewSubmitted'],
      entities: ['Product', 'Category', 'Review'],
      services: ['CatalogService', 'InventoryService'],
    },
    order: {
      events: ['ProductSold', 'PaymentProcessed', 'OrderShipped'],
      entities: ['Order', 'Payment', 'Shipment'],
      services: ['CheckoutService', 'PaymentService'],
    },
  };

```

### 3. Strangler Fig Pattern

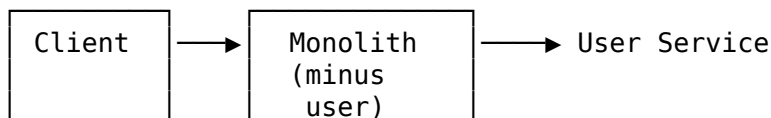
#### 3.1 Konsep

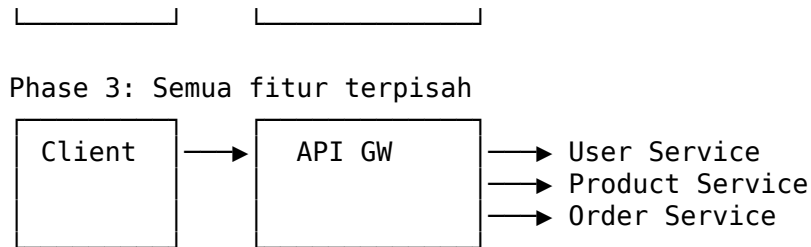
Strangler Fig — pohon yang tumbuh melilit pohon inang sampai inangnya mati. Dalam software: fitur baru dibangun sebagai service terpisah, lalu routing dialihkan perlahan.

Phase 1: Monolith melayani semua



Phase 2: Satu fitur dipindah





Phase 3: Semua fitur terpisah

### 3.2 Route-by-Route Migration

```

// Monolith – sebelum refactor
app.use('/api/users', userRoutes);    // akan dipindah
app.use('/api/products', productRoutes); // masih di monolith
app.use('/api/orders', orderRoutes);  // masih di monolith

// Langkah 1: Proxy di monolith untuk route yang sudah dipindah
app.use('/api/users', createProxy('http://user-service:3001'));
// Request /api/users diteruskan ke service baru

// Langkah 2: Route baru langsung ke service baru
// Monolith hanya jadi proxy router

// Langkah 3: Matikan monolith, API Gateway ambil alih
  
```

### 3.3 Implementation Pattern

```

// strangler-proxy.ts – proxy di monolith
import httpProxy from 'http-proxy-middleware';

const migratedRoutes = {
  '/api/users': 'http://user-service:3001',
  '/api/auth': 'http://user-service:3001',
};

for (const [route, target] of Object.entries(migratedRoutes)) {
  app.use(route, createProxy({
    target,
    changeOrigin: true,
    // log migration status
    on: {
      proxyReq: (proxyReq) => console.log(`[Strangler] ${proxyReq.path} -> ${target}`, proxyReq),
      proxyRes: (proxyRes) => proxyRes.headers['x-migrated'] = 'true',
    }
  }));
}
  
```

```
}
```

---

## 4. Shared Kernel vs Anti-Corruption Layer

### 4.1 Shared Kernel

Bagian kode yang **dipakai bersama** antar context. Minimal dan stabil.

```
// shared-kernel/types.ts – tipe dasar yang dipakai semua service
export type UUID = string;
export type Email = string;
export type Timestamp = string; // ISO 8601
```

```
export interface BaseEntity {
  id: UUID;
  createdAt: Timestamp;
  updatedAt: Timestamp;
}
```

```
export interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: string;
  correlationId: string;
}
```

**Aturan Shared Kernel:** - Hanya tipe dasar — tidak ada business logic - Jarang berubah (stable) - Semua service depend on it

### 4.2 Anti-Corruption Layer (ACL)

Lapisan yang **melindungi** context dari perubahan di context lain. Berguna saat integrasi dengan legacy system.

```
// order-service/acl/user-acl.ts
// ACL melindungi Order Service dari model User yang berubah-ubah
```

```
interface UserResponse {
  id: string;
  name: string;
  email: string;
  role: string;
}
```

```

// ACL – translate dari user service model ke order context model
export class UserAcl {
  async getUser(userId: string): Promise<UserResponse> {
    const raw = await this.userServiceClient.getUser(userId);

    // translate ke format yang dipahami Order Context
    return {
      id: raw.id,
      name: `${raw.firstName} ${raw.lastName}`, // gabung field
      email: raw.email,
      role: raw.role === 'premium' ? 'VIP' : 'Regular', // translate enum
    };
  }
}

```

**Kapan pake ACL:** - Integrasi dengan legacy system yang modelnya beda - Context external yang sering berubah - Butuh isolation kuat antar tim

---

## 5. Service Boundaries — Studi Kasus

### 5.1 User Service

```

// Boundaries:
// - Registration & authentication
// - Profile management
// - Role & permission

interface User {
  id: UUID;
  email: Email;
  passwordHash: string;
  profile: {
    name: string;
    avatar?: string;
  };
  role: 'customer' | 'admin';
  createdAt: Timestamp;
}

// Owns: users table
// Exposes: POST /register, POST /login, GET /profile, PUT /profile
// Depends on: nothing (auth sendiri)

```

## 5.2 Product Service

```
// Boundaries:  
// - Product catalog  
// - Inventory management  
// - Categories & search  
  
interface Product {  
    id: UUID;  
    sku: string;  
    name: string;  
    description: string;  
    price: number;  
    stock: number;  
    categoryId: UUID;  
    reviews: Review[];  
    createdAt: Timestamp;  
}  
  
// Owns: products, categories, reviews tables  
// Exposes: CRUD products, search, stock check  
// Depends on: User Service (for seller info, via ACL)
```

## 5.3 Order Service

```
// Boundaries:  
// - Shopping cart  
// - Checkout process  
// - Payment processing  
// - Shipping & tracking  
  
interface Order {  
    id: UUID;  
    userId: UUID;  
    items: OrderItem[];  
    total: number;  
    status: 'pending' | 'paid' | 'shipped' | 'delivered';  
    payment: PaymentInfo;  
    shipping: ShippingInfo;  
    createdAt: Timestamp;  
}  
  
interface OrderItem {  
    productId: UUID;  
    quantity: number;  
    price: number;
```

```
}
```

```
// Owns: orders, payments, shipments tables  
// Exposes: POST /checkout, GET /orders, POST /pay  
// Depends on: Product Service (for price/stock), User Service (for address)
```

---

## 6. Latihan: Identify Bounded Contexts

### Studi Kasus: Platform Learning Management System (LMS)

Sebuah LMS monolith ingin dipecah jadi microservices. Fitur-fiturnya:

1. **Manajemen Kursus** — CRUD kursus, materi, quiz
2. **Manajemen Pengguna** — daftar, login, profil, roles (siswa, instruktur, admin)
3. **Enrollment** — daftar kursus, progres belajar, sertifikat
4. **Pembayaran** — checkout kursus, refund, invoice
5. **Forum Diskusi** — tanya-jawab per kursus, komentar

### Tugas

1. Identifikasi minimal 3 bounded context
2. Tentukan entities utama per context
3. Tentukan service boundary — apa yang di-own dan apa yang di-expose
4. Identifikasi shared kernel (tipe data yang dipakai bersama)
5. Apakah perlu Anti-Corruption Layer? Dimana?

### Template Jawaban

## Bounded Context 1: [Nama Context]

- Entities: ...
- Owns: ...
- Exposes: ...
- Depends on: ...

## Bounded Context 2: [Nama Context]

- Entities: ...
- Owns: ...
- Exposes: ...
- Depends on: ...

## Shared Kernel:

- ...

---

## □ Referensi

- [Domain-Driven Design — Eric Evans](#)
- [Strangler Fig Pattern — Martin Fowler](#)
- [Bounded Context — Martin Fowler](#)
- [Anti-Corruption Layer](#)

## Sesi 02: Docker Multi-Service

**Durasi:** 120 menit

**Prerequisites:** Modul 21 (Docker dasar)

---

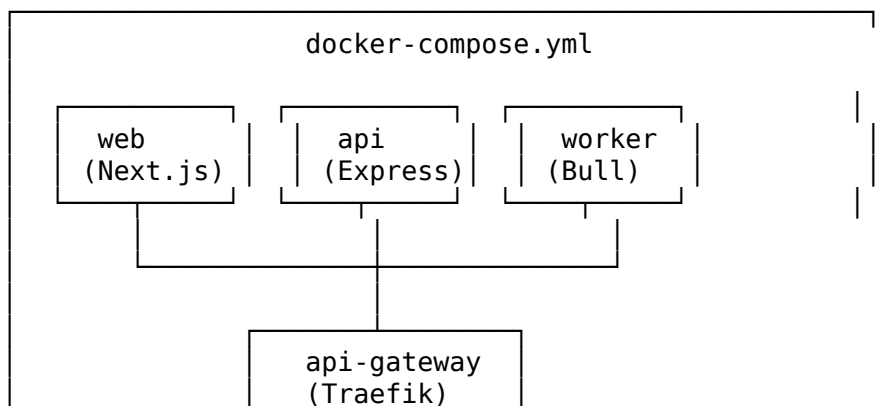
## □ Tujuan

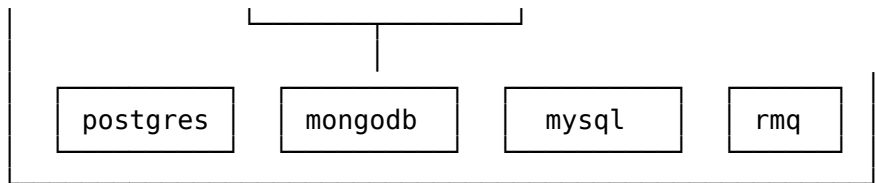
- Membangun Docker Compose multi-service
  - Mengelola network isolation antar service
  - Konfigurasi environment per service
  - Volume mounts untuk persistent data
  - Health checks & restart policy
- 

## 1. Docker Compose Multi-Service

### 1.1 Arsitektur Layanan

Kita akan menjalankan 4 service:





## 1.2 docker-compose.yml — Struktur Dasar

```
# docker-compose.yml
version: '3.8'

services:
  # =====
  # User Service
  # =====
  user-service:
    build:
      context: ./services/user-service
      dockerfile: Dockerfile
    container_name: ms-user-service
    ports:
      - "3001:3001"
    environment:
      - NODE_ENV=production
      - PORT=3001
      - DB_HOST=user-db
      - DB_PORT=5432
      - DB_NAME=users
      - DB_USER=user_service
      - DB_PASSWORD=${USER_DB_PASSWORD}
      - JWT_SECRET=${JWT_SECRET}
      - RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
    env_file:
      - ./services/user-service/.env
    depends_on:
      user-db:
        condition: service_healthy
      rabbitmq:
        condition: service_started
    volumes:
      - user-service-logs:/var/log/app
    networks:
      - backend
      - database
```

```

restart: unless-stopped
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:3001/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 40s

user-db:
  image: postgres:16-alpine
  container_name: ms-user-db
  environment:
    - POSTGRES_DB=users
    - POSTGRES_USER=user_service
    - POSTGRES_PASSWORD=${USER_DB_PASSWORD}
  volumes:
    - user-db-data:/var/lib/postgresql/data
    - ./services/user-service/init.sql:/docker-entrypoint-initdb.d/init.sql
  networks:
    - database
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U user_service -d users"]
    interval: 10s
    timeout: 5s
    retries: 5
    restart: unless-stopped

# =====
# Product Service
# =====
product-service:
  build:
    context: ./services/product-service
    dockerfile: Dockerfile
  container_name: ms-product-service
  ports:
    - "3002:3002"
  environment:
    - NODE_ENV=production
    - PORT=3002
    - DB_HOST=product-db
    - DB_PORT=27017
    - DB_NAME=products
    - RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
  env_file:
    - ./services/product-service/.env

```

```

depends_on:
  product-db:
    condition: service_healthy
  rabbitmq:
    condition: service_started
networks:
  - backend
  - database
restart: unless-stopped

product-db:
  image: mongo:7
  container_name: ms-product-db
  environment:
    - MONGO_INITDB_DATABASE=products
  volumes:
    - product-db-data:/data/db
  networks:
    - database
  healthcheck:
    test: echo 'db.runCommand("ping").ok' | mongosh --quiet
    interval: 10s
    timeout: 5s
    retries: 5
  restart: unless-stopped

# =====
# Order Service
# =====
order-service:
  build:
    context: ./services/order-service
    dockerfile: Dockerfile
  container_name: ms-order-service
  ports:
    - "3003:3003"
  environment:
    - NODE_ENV=production
    - PORT=3003
    - DB_HOST=order-db
    - DB_PORT=3306
    - DB_NAME=orders
    - DB_USER=order_service
    - DB_PASSWORD=${ORDER_DB_PASSWORD}
    - RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672
  env_file:

```

```

    - ./services/order-service/.env
depends_on:
  order-db:
    condition: service_healthy
  rabbitmq:
    condition: service_started
networks:
  - backend
  - database
restart: unless-stopped

order-db:
  image: mysql:8
  container_name: ms-order-db
  environment:
    - MYSQL_DATABASE=orders
    - MYSQL_USER=order_service
    - MYSQL_PASSWORD=${ORDER_DB_PASSWORD}
    - MYSQL_RANDOM_ROOT_PASSWORD=yes
  volumes:
    - order-db-data:/var/lib/mysql
  networks:
    - database
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s
    timeout: 5s
    retries: 5
    restart: unless-stopped

# =====
# Message Queue
# =====
rabbitmq:
  image: rabbitmq:3.13-management-alpine
  container_name: ms-rabbitmq
  ports:
    - "5672:5672" # AMQP
    - "15672:15672" # Management UI
  environment:
    - RABBITMQ_DEFAULT_USER=guest
    - RABBITMQ_DEFAULT_PASS=guest
  volumes:
    - rabbitmq-data:/var/lib/rabbitmq
  networks:
    - message-queue

```

```

healthcheck:
  test: ["CMD", "rabbitmq-diagnostics", "check_port_connectivity"]
  interval: 15s
  timeout: 10s
  retries: 5
  restart: unless-stopped

# =====
# API Gateway (Traefik)
# =====
gateway:
  image: traefik:v3.0
  container_name: ms-gateway
  ports:
    - "80:80"          # HTTP
    - "443:443"        # HTTPS
    - "8080:8080"      # Dashboard
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - ./gateway/traefik.yml:/etc/traefik/traefik.yml
    - ./gateway/dynamic.yml:/etc/traefik/dynamic.yml
  networks:
    - backend
    - frontend
  depends_on:
    user-service:
      condition: service_started
    product-service:
      condition: service_started
    order-service:
      condition: service_started
  restart: unless-stopped

# =====
# Networks
# =====
networks:
  frontend:
    driver: bridge
    name: ms-frontend
  backend:
    driver: bridge
    internal: false
    name: ms-backend
  database:
    driver: bridge

```

```

    internal: true      # Database inaccessible dari luar
    name: ms-database
message-queue:
  driver: bridge
  internal: true      # RabbitMQ internal-only
  name: ms-queue

# =====
# Volumes
# =====
volumes:
  user-db-data:
    name: ms-user-db-data
  product-db-data:
    name: ms-product-db-data
  order-db-data:
    name: ms-order-db-data
  rabbitmq-data:
    name: ms-rabbitmq-data
  user-service-logs:
    name: ms-user-logs

```

---

## 2. Network Isolation

### 2.1 Jenis Network

| Network       | Driver | Internal   | Akses              | Service di Dalamnya                                                         |
|---------------|--------|------------|--------------------|-----------------------------------------------------------------------------|
| frontend      | bridge | no         | Public             | gateway                                                                     |
| backend       | bridge | no         | Service-to-service | user-service, product-service, order-service, gateway                       |
| database      | bridge | <b>yes</b> | Service DB only    | user-db, product-db, order-db, user-service, product-service, order-service |
| message-queue | bridge | <b>yes</b> | Queue only         | rabbitmq, semua service                                                     |

## 2.2 Kenapa Internal Network?

```
# Database dan Queue TIDAK perlu akses dari luar
database:
  internal: true    # Container di network ini cuma bisa diakses dari
                   # container lain yang juga di network yang sama

# Tanpa internal: database bisa diakses dari host → risk langsung dari luar
# Dengan internal: database cuma bisa diakses via service layer
```

## 2.3 Service Terhubung ke Multiple Network

```
# user-service ada di 2 network: backend + database
services:
  user-service:
    networks:
      - backend    # → komunikasi REST antar service + gateway
      - database   # → akses ke user-db (PostgreSQL)

# Ini penting: database di internal network, tapi service bisa akses
# karena mereka satu network. Tapi dari host/luar ga bisa.
```

---

## 3. Environment per Service

### 3.1 Tiga Level Konfigurasi

```
services:
  user-service:
    # Level 1: environment langsung (override .env file)
    environment:
      - NODE_ENV=production
      - PORT=3001
      - DB_HOST=user-db

    # Level 2: env_file (default values, versioned)
    env_file:
      - ./services/user-service/.env

    # Level 3: .env file project-level (rahasia, tidak versioned)
    # Di root project: .env – docker-compose auto-load
```

### 3.2 .env File per Service

```
# services/user-service/.env – versioned, default values
DB_PORT=5432
DB_NAME=users
DB_USER=user_service
REDIS_URL=redis://redis:6379
LOG_LEVEL=info

# services/user-service/.env.local – NOT versioned, actual secrets
DB_PASSWORD=s3cr3t_local_dev_only
JWT_SECRET=change_me_in_production
```

### 3.3 Variable Substitution

```
# Root .env – shared across all services
USER_DB_PASSWORD=dev_password_123
ORDER_DB_PASSWORD=dev_password_456
JWT_SECRET=dev_jwt_secret

# docker-compose.yml – referensi ke root .env
services:
  user-service:
    environment:
      - DB_PASSWORD=${USER_DB_PASSWORD}
  order-service:
    environment:
      - DB_PASSWORD=${ORDER_DB_PASSWORD}
```

---

## 4. Volume Mounts

### 4.1 Persistent Data

```
# Data yang TIDAK boleh hilang saat container restart
volumes:
  user-db-data: # named volume – dikelola Docker
    name: ms-user-db-data

  product-db-data:
    name: ms-product-db-data

services:
  user-db:
    volumes:
      - user-db-data:/var/lib/postgresql/data # data PostgreSQL
```

## 4.2 Config Mounts (Bind Mount)

```
services:
  gateway:
    volumes:
      # Config file dari host ke container (bind mount)
      - ./gateway/traefik.yml:/etc/traefik/traefik.yml:ro
      - ./gateway/dynamic.yml:/etc/traefik/dynamic.yml:ro

      # Docker socket – Traefik butuh untuk service discovery
      - /var/run/docker.sock:/var/run/docker.sock:ro
```

## 4.3 Init Scripts

```
services:
  user-db:
    volumes:
      # SQL init – dijalankan sekali saat pertama container start
      - ./services/user-service/init.sql:/docker-entrypoint-initdb.d/init.sql
```

## 4.4 Log Volume

```
services:
  user-service:
    volumes:
      - user-service-logs:/var/log/app # log persist meski container restart

volumes:
  user-service-logs:
    name: ms-user-logs
```

---

# 5. Health Checks & Restart Policy

## 5.1 Health Check per Service

```
services:
  user-db:
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user_service -d users"]
      interval: 10s # cek tiap 10 detik
      timeout: 5s # timeout 5 detik
      retries: 5 # after 5 gagal → mark unhealthy
      start_period: 30s # grace period sebelum mulai cek
```

```

user-service:
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:3001/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 40s

```

## 5.2 depends\_on with Condition

```

services:
  user-service:
    depends_on:
      user-db:
        condition: service_healthy  # Tunggu sampai DB sehat
      rabbitmq:
        condition: service_started  # Tunggu container start (tidak harus sehat)

```

**Catatan:** condition: service\_healthy baru tersedia di Compose v3.8+. Docker Compose v2+ support ini.

## 5.3 Restart Policy

```

services:
  user-service:
    restart: unless-stopped
# Nilai lain:
# "no"          - tidak restart otomatis (default)
# "always"      - selalu restart, termasuk setelah docker restart
# "on-failure"  - restart hanya jika exit code != 0
# "unless-stopped" - restart kecuali dihentikan manual

```

## 5.4 Healthcheck di Dockerfile

```

# services/user-service/Dockerfile
FROM node:20-alpine

# Install curl untuk healthcheck
RUN apk add --no-cache curl

WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .

EXPOSE 3001

```

```
CMD ["node", "dist/server.js"]
```

```
# HEALTHCHECK di Dockerfile (alternatif dari compose)
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD curl -f http://localhost:3001/health || exit 1
```

## 6. Latihan: docker-compose dengan 4 Service

### Studi Kasus

Buat docker-compose.yml untuk sistem **Blog Platform** dengan:

| Service             | Teknologi            | Port | Database   |
|---------------------|----------------------|------|------------|
| Blog API            | Express + TypeScript | 4000 | PostgreSQL |
| Comment API         | Express + TypeScript | 4001 | MongoDB    |
| Notification Worker | Node.js + Bull       | -    | Redis      |
| API Gateway         | Traefik              | 80   | -          |

### Spesifikasi

1. **Blog API** — CRUD artikel, depend on: postgres (healthy), redis (started)
2. **Comment API** — CRUD komentar, depend on: mongo (healthy)
3. **Notification Worker** — process queue dari Redis, depend on: redis (healthy)
4. **API Gateway** — routing /api/blog→Blog API, /api/comment→Comment API

### Persyaratan

- Gunakan 3 network: frontend (public), backend (service-to-service), data (internal)
- Named volumes untuk semua database
- Health check untuk setiap database
- Restart policy unless-stopped
- Environment variable via .env file per service
- Bagian comment memiliki volume untuk MongoDB init script

### Template Jawaban

```
# docker-compose.yml
version: '3.8'
```

```
services:
  blog-api:
    # ...

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
  data:
    driver: bridge
    internal: true

volumes:
  postgres-data:
  mongo-data:
```

---

## □ Referensi

- [Docker Compose Overview](#)
- [Docker Networking Overview](#)
- [Health Check in Docker](#)
- [Compose File Version 3 Reference](#)

## Sesi 03: Inter-Service Communication

**Durasi:** 120 menit

**Prerequisites:** Sesi 01, Sesi 02

---

## □ Tujuan

- Merancang REST API internal antar service
  - Implementasi gRPC untuk komunikasi performa tinggi
  - Menggunakan message queue (RabbitMQ) untuk event-driven
  - Menerapkan Saga pattern untuk distributed transaction
  - Menangani failure dengan timeout, retry, circuit breaker
-

## 1. REST — Internal API Design

### 1.1 Internal vs Public API

```
// ❗ JANGAN: Service internal pake API yang sama dengan public
// Public API: /api/users – lengkap dengan auth, rate limit, dll
// Internal API: /internal/users – langsung, tanpa overhead
```

```
// ❗ PISAHKAN: Internal endpoint
// user-service
router.get('/internal/users/:id', async (req, res) => {
  const user = await userService.findById(req.params.id);
  // Langsung return – tanpa auth check (sudah di-handle caller)
  res.json({ success: true, data: user });
});

router.get('/users/:id', authenticate, async (req, res) => {
  // Public endpoint – dengan auth, validasi, rate limit
  const user = await userService.findById(req.params.id);
  res.json({ success: true, data: user });
});
```

### 1.2 Timeout & Retry

```
// services/order-service/src/clients/user-client.ts
import axios from 'axios';
import axiosRetry from 'axios-retry';

const userClient = axios.create({
  baseURL: 'http://user-service:3001',
  timeout: 5000, // 5 detik – jangan sampai nunggu lama
  headers: {
    'X-Internal-Call': 'true',
    'X-Service-Name': 'order-service',
  },
});

// Retry logic – 3 kali, exponential backoff
axiosRetry(userClient, {
  retries: 3,
  retryDelay: (retryCount) => {
    return Math.pow(2, retryCount) * 200; // 400ms, 800ms, 1600ms
  },
  retryCondition: (error) => {
    // Retry cuma untuk transient errors
    return [
```

```

        'ECONNABORTED', // timeout
        'ECONNRESET',   // connection reset
        'ERR_NETWORK',  // network error
    ].includes(error.code || '')
    || (error.response?.status ?? 0) >= 500; // server error
  },
});

```

### 1.3 Circuit Breaker

*// circuit-breaker.ts – Opossum implementation*

```
import CircuitBreaker from 'opossum';
```

```

const options = {
  timeout: 3000, // Timeout per request: 3 detik
  errorThresholdPercentage: 50, // Jika 50% request gagal → open circuit
  resetTimeout: 30000, // Coba lagi setelah 30 detik
  name: 'user-service',
};

```

```

const breaker = new CircuitBreaker(async (userId: string) => {
  const response = await userClient.get(`/internal/users/${userId}`);
  return response.data;
}, options);

```

*// Event listeners*

```

breaker.on('open', () => console.warn('[CB] Circuit OPEN – user-service down'));
breaker.on('halfOpen', () => console.info('[CB] Circuit HALF-OPEN – testing...'));
breaker.on('close', () => console.info('[CB] Circuit CLOSED – user-service back'));

```

*// Fallback – kalau circuit open*

```

breaker.fallback(() => ({
  success: true,
  data: { id: 'unknown', name: 'Unknown User', email: 'unknown@fallback' },
  fromCache: true,
}));

```

*// Usage*

```

async function getUser(userId: string) {
  try {
    const result = await breaker.fire(userId);
    return result.data;
  } catch (error) {
    console.error('[CB] All retries exhausted', error);
    throw new Error('User service unavailable');
  }
}

```

```
}  
}
```

## 1.4 Correlation ID

```
// correlation-id.ts – trace request antar service  
import { v4 as uuidv4 } from 'uuid';  
  
// Middleware: inject correlation ID dari request masuk  
function correlationMiddleware(req: any, res: any, next: any) {  
  const correlationId = req.headers['x-correlation-id'] || uuidv4();  
  req.correlationId = correlationId;  
  res.setHeader('x-correlation-id', correlationId);  
  
  // Propagate ke downstream calls  
  req.downstreamHeaders = {  
    'X-Correlation-Id': correlationId,  
    'X-Service-Name': process.env.SERVICE_NAME || 'unknown',  
  };  
  
  next();  
}  
  
// Axios interceptor: attach correlation ID ke semua outgoing request  
axiosInstance.interceptors.request.use((config) => {  
  const correlationId = getCurrentCorrelationId(); // dari AsyncLocalStorage  
  config.headers['X-Correlation-Id'] = correlationId;  
  return config;  
});
```

---

## 2. gRPC — High-Performance Communication

### 2.1 Protobuf Service Definition

```
// proto/user.proto  
syntax = "proto3";  
  
package user;  
  
service UserService {  
  rpc GetUser (GetUserRequest) returns (User);  
  rpc ListUsers (ListUsersRequest) returns (ListUsersResponse);  
  rpc CreateUser (CreateUserRequest) returns (User);  
  rpc UpdateUser (UpdateUserRequest) returns (User);  
}
```

```

    // Server streaming: kirim list user satu per satu
    rpc StreamUsers (StreamUsersRequest) returns (stream User);

    // Bidirectional streaming: chat antar service
    rpc Chat (stream ChatMessage) returns (stream ChatMessage);
}

message GetUserRequest {
    string id = 1;
}

message User {
    string id = 1;
    string name = 2;
    string email = 3;
    string role = 4;
    int64 created_at = 5; // Unix timestamp
}

message ListUsersRequest {
    int32 page = 1;
    int32 limit = 2;
    string role_filter = 3;
}

message ListUsersResponse {
    repeated User users = 1;
    int32 total = 2;
    int32 page = 3;
}

message StreamUsersRequest {
    string role = 1;
}

message ChatMessage {
    string from_service = 1;
    string to_service = 2;
    string message = 3;
    int64 timestamp = 4;
}

```

## 2.2 gRPC Server (TypeScript)

```
// services/user-service/src/grpc/server.ts
import * as grpc from '@grpc/grpc-js';
import * as protoLoader from '@grpc/proto-loader';
import { UserService } from '../services/user-service';

const PROTO_PATH = __dirname + '/../..../proto/user.proto';

const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true,
});

const userProto = grpc.loadPackageDefinition(packageDefinition).user as any;

function startGrpcServer() {
  const server = new grpc.Server();

  server.addService(userProto.UserService.service, {
    GetUser: async (call: any, callback: any) => {
      try {
        const user = await UserService.findById(call.request.id);
        callback(null, user);
      } catch (error) {
        callback({
          code: grpc.status.NOT_FOUND,
          message: `User ${call.request.id} not found`,
        });
      }
    },
  },
  {
    ListUsers: async (call: any, callback: any) => {
      const { page, limit, role_filter } = call.request;
      const result = await UserService.findAll({ page, limit, role: role_filter });
      callback(null, result);
    },
  },
  {
    // Server streaming
    StreamUsers: (call: any) => {
      const { role } = call.request;
      const stream = UserService.streamUsers(role);
    },
  },
}
```

```

    stream.on('data', (user: any) => {
      call.write(user); // kirim satu per satu
    });

    stream.on('end', () => {
      call.end(); // selesai streaming
    });
  },

  // Bidirectional streaming
  Chat: (call: any) => {
    call.on('data', (message: any) => {
      console.log(`[Chat] ${message.from_service} -> ${message.to_service}: ${message}`);

      // Echo back – atau forward ke service lain
      call.write({
        ...message,
        timestamp: Date.now(),
      });
    });

    call.on('end', () => call.end());
  },
});

const address = `0.0.0.0:${process.env.GRPC_PORT || 50051}`;
server.bindAsync(address, grpc.ServerCredentials.createInsecure(), () => {
  console.log(`[gRPC] Server listening on ${address}`);
  server.start();
});
}

```

## 2.3 gRPC Client

```

// services/order-service/src/clients/user-grpc-client.ts
import * as grpc from '@grpc/grpc-js';
import * as protoLoader from '@grpc/proto-loader';

const PROTO_PATH = __dirname + '../../../proto/user.proto';
const packageDefinition = protoLoader.loadSync(PROTO_PATH, {
  keepCase: true,
  longs: String,
  enums: String,
  defaults: true,
  oneofs: true,
});

```

```

});

const userProto = grpc.loadPackageDefinition(packageDefinition).user as any;

class UserGrpcClient {
  private client: any;

  constructor() {
    this.client = new userProto.UserService(
      'user-service:50051',
      grpc.credentials.createInsecure()
    );
  }

  // Unary call – mirip REST
  async getUser(id: string): Promise<any> {
    return new Promise((resolve, reject) => {
      this.client.GetUser({ id }, (error: any, response: any) => {
        if (error) reject(error);
        else resolve(response);
      });
    });
  }

  // Server streaming – dengerin stream
  streamUsers(role?: string): AsyncIterable<any> {
    const call = this.client.StreamUsers({ role: role || '' });

    return {
      [Symbol.asyncIterator]: () => ({
        next: () => new Promise((resolve) => {
          call.on('data', (user: any) => resolve({ value: user, done: false }));
          call.on('end', () => resolve({ value: undefined, done: true }));
        }),
      }),
    };
  }

  // Bidirectional streaming
  createChat() {
    const call = this.client.Chat();

    return {
      send: (toService: string, message: string) => {
        call.write({
          from_service: 'order-service',

```

```

        to_service: toService,
        message,
        timestamp: Date.now(),
    });
},
onMessage: (handler: Function) => {
    call.on('data', handler);
},
close: () => call.end(),
};
}
}

```

---

### 3. Message Queue — RabbitMQ

#### 3.1 Kapan Pakai Message Queue

| Situasi                                  | Pakai REST?              | Pakai Queue?             |
|------------------------------------------|--------------------------|--------------------------|
| Butuh response langsung (get user)       | <input type="checkbox"/> | <input type="checkbox"/> |
| Butuh reliability (order harus diproses) | <input type="checkbox"/> | <input type="checkbox"/> |
| Satu event → banyak listener             | <input type="checkbox"/> | <input type="checkbox"/> |
| Butuh retry & dead letter                | <input type="checkbox"/> | <input type="checkbox"/> |
| Latency rendah                           | <input type="checkbox"/> | <input type="checkbox"/> |

#### 3.2 RabbitMQ Setup

```

// shared/message-bus.ts
import amqp from 'amqplib';

class MessageBus {
    private connection!: amqp.Connection;
    private channel!: amqp.Channel;

    async connect(url: string = process.env.RABBITMQ_URL || 'amqp://localhost') {
        this.connection = await amqp.connect(url);
        this.channel = await this.connection.createChannel();

        // Graceful shutdown
        process.on('SIGINT', () => this.close());
        process.on('SIGTERM', () => this.close());
    }
}

```

```

async close() {
    await this.channel?.close();
    await this.connection?.close();
}

// =====
// Exchange & Queue
// =====

async setupExchanges() {
    // Direct exchange: routing by exact routing key
    await this.channel.assertExchange('orders.direct', 'direct', { durable: true })

    // Topic exchange: routing by pattern
    await this.channel.assertExchange('events.topic', 'topic', { durable: true })

    // Fanout exchange: broadcast to all queues
    await this.channel.assertExchange('notifications.fanout', 'fanout', { durable: true })
}

async setupQueues() {
    // Order queues
    await this.channel.assertQueue('orders.create', { durable: true });
    await this.channel.assertQueue('orders.payment', { durable: true });
    await this.channel.assertQueue('orders.shipping', { durable: true });

    // Dead letter queue
    await this.channel.assertQueue('orders.dlq', { durable: true });

    // Bind queues
    await this.channel.bindQueue('orders.create', 'orders.direct', 'order.create');
    await this.channel.bindQueue('orders.payment', 'orders.direct', 'order.paid');
    await this.channel.bindQueue('orders.shipping', 'orders.direct', 'order.shipping');
}

// =====
// Publish
// =====

async publish<T>(exchange: string, routingKey: string, message: T) {
    const correlationId = getCurrentCorrelationId();

    this.channel.publish(
        exchange,
        routingKey,
        Buffer.from(JSON.stringify(message)),
    )
}

```

```

    {
      persistent: true,
      headers: {
        'x-correlation-id': correlationId,
        'x-service': process.env.SERVICE_NAME,
      },
    }
  );

  console.log(`[MQ] Published ${routingKey} (correlationId: ${correlationId})`
}

// =====
// Consume
// =====

async consume(queue: string, handler: (msg: any) => Promise<void>) {
  await this.channel.consume(queue, async (msg) => {
    if (!msg) return;

    try {
      const content = JSON.parse(msg.content.toString());
      console.log(`[MQ] Consumed ${msg.fields.routingKey}`);

      await handler(content);

      // Acknowledge – message sukses diproses
      this.channel.ack(msg);
    } catch (error) {
      console.error(`[MQ] Error processing ${msg.fields.routingKey}:`, error);

      // Reject – kirim ke DLQ setelah 3 retry
      const retryCount = (msg.properties.headers?.['x-retry-count'] || 0) + 1;

      if (retryCount >= 3) {
        console.warn(`[MQ] Sending to DLQ after ${retryCount} retries`);
        this.channel.reject(msg, false); // false = no requeue → masuk DLQ
      } else {
        // Retry dengan delay
        const delayMs = Math.pow(2, retryCount) * 1000;
        setTimeout(() => {
          this.channel.publish(
            msg.fields.exchange,
            msg.fields.routingKey,
            msg.content,
            { headers: { 'x-retry-count': retryCount } }
          );
        }, delayMs);
      }
    }
  });
}

```

```

    );
    this.channel.ack(msg); // ack yang lama
  }, delayMs);
}
}
});
}
}

```

```
export const messageBus = new MessageBus();
```

### 3.3 Event Schema

*// shared/events.ts – semua event dalam sistem*

*// Setiap event punya schema yang strict*

```

interface BaseEvent {
  id: string;           // UUID – unique
  type: string;         // Event type
  source: string;       // Service name
  correlationId: string;
  timestamp: number;    // Unix ms
  version: number;      // Schema version
}

```

*// User Events*

```

interface UserCreatedEvent extends BaseEvent {
  type: 'user.created';
  data: {
    userId: string;
    email: string;
    name: string;
    role: 'customer' | 'admin';
  };
}

```

*// Product Events*

```

interface ProductStockChangedEvent extends BaseEvent {
  type: 'product.stock.changed';
  data: {
    productId: string;
    oldStock: number;
    newStock: number;
    change: number; // negatif = sold
  };
}

```

```

}

// Order Events
interface OrderCreatedEvent extends BaseEvent {
  type: 'order.created';
  data: {
    orderId: string;
    userId: string;
    items: { productId: string; quantity: number; price: number }[];
    total: number;
  };
}

interface OrderPaidEvent extends BaseEvent {
  type: 'order.paid';
  data: {
    orderId: string;
    paymentId: string;
    amount: number;
    paidAt: string;
  };
}

type AppEvent = UserCreatedEvent | ProductStockChangedEvent | OrderCreatedEvent

```

### 3.4 Idempotency

```

// idempotency.ts – pastikan event ga diproses 2x

class IdempotencyStore {
  private processed = new Set<string>();
  private ttl = 3600000; // 1 jam

  async isProcessed(eventId: string): Promise<boolean> {
    // Redis: SETNX eventId → 1 EX 3600
    const result = await redis.set(eventId, '1', 'NX', 'EX', this.ttl / 1000);
    return result === null; // null = sudah ada sebelumnya
  }
}

// Usage di consumer
async function handleOrderCreated(event: OrderCreatedEvent) {
  if (await idempotency.isProcessed(event.id)) {
    console.log(`[Idempotency] Event ${event.id} already processed, skipping`);
    return;
  }
}

```

```

    }

    // Process event ...
    await processOrder(event.data);
  }

```

### 3.5 Outbox Pattern

```

// outbox.ts – atomic DB write + event publish

// Problem: DB write sukses, tapi event gagal publish
// Atau: Event terkirim, tapi DB write gagal
// → Outbox pattern solves this

async function createOrder(orderData: CreateOrderDTO) {
  const transaction = await sequelize.transaction();

  try {
    // Step 1: Insert order ke database
    const order = await Order.create(orderData, { transaction });

    // Step 2: Insert event ke outbox table (table yang sama!)
    await OutboxEvent.create({
      id: uuidv4(),
      type: 'order.created',
      payload: order.toJSON(),
      status: 'pending',
      createdAt: new Date(),
    }, { transaction });

    // Step 3: Commit – DB write + event write atomic
    await transaction.commit();

    // Step 4: Publish event (diluar transaction – bisa gagal, ga masalah)
    try {
      await messageBus.publish('orders.direct', 'order.created', order.toJSON());

      // Update outbox status jadi published
      await OutboxEvent.update(
        { status: 'published', publishedAt: new Date() },
        { where: { type: 'order.created', status: 'pending' } }
      );
    } catch (publishError) {
      // Event gagal publish – outbox masih pending
      // Outbox publisher akan retry nanti
    }
  }
}

```

```

        console.error('[Outbox] Publish failed, will retry:', publishError);
    }

    return order;
} catch (error) {
    await transaction.rollback();
    throw error;
}
}

// Outbox Publisher – cron job yang publish ulang event pending
async function processOutbox() {
    const pendingEvents = await OutboxEvent.findAll({
        where: { status: 'pending' },
        limit: 100,
    });

    for (const event of pendingEvents) {
        try {
            await messageBus.publish('orders.direct', event.type, event.payload);
            event.status = 'published';
            event.publishedAt = new Date();
            await event.save();
        } catch (error) {
            event.retryCount += 1;
            event.lastError = (error as Error).message;
            await event.save();

            if (event.retryCount >= 10) {
                event.status = 'failed';
                await event.save();
            }
        }
    }
}

```

---

## 4. Saga Pattern

### 4.1 Choreography Saga

Setiap service publish event, service lain react. No central coordinator.

Skenario: User checkout order

1. Order Service: create order → publish "OrderCreated"
2. Payment Service: listen "OrderCreated" → process payment → publish "PaymentProce"
3. Inventory Service: listen "PaymentProcessed" → reduce stock → publish "StockUpda"
4. Shipping Service: listen "StockUpdated" → schedule shipping → publish "OrderShip"
5. Notification Service: listen "OrderShipped" → send email

Jika Payment gagal:

2. Payment Service: publish "PaymentFailed"
3. Order Service: listen "PaymentFailed" → cancel order
4. Inventory Service: listen "PaymentFailed" → tidak perlu rollback (stock belum di

*// order-service/saga/order-saga.ts – Choreography*

```
class OrderSaga {
  async start(orderData: OrderData) {
    const order = await Order.create(orderData);

    // Publish event – service lain yang akan react
    await messageBus.publish('orders.direct', 'order.created', {
      orderId: order.id,
      userId: order.userId,
      items: order.items,
      total: order.total,
    });

    // Listen untuk compensation events
    await messageBus.consume('orders.payment-failed', async (event) => {
      // Compensation: cancel order
      await Order.update(
        { status: 'cancelled', cancelReason: 'Payment failed' },
        { where: { id: event.data.orderId } }
      );
    });
  }
}
```

## 4.2 Orchestration Saga

Satu service (Orchestrator) yang mengontrol flow. Setiap service hanya menjalankan perintah.

Skenario: User checkout order

Orchestrator (Order Service):

1. → Order Service: create order (pending)
2. → Payment Service: process payment
3. □ Payment gagal → Order Service: cancel order (compensation)

4. ✓ Payment sukses → → Inventory Service: reduce stock
5. □ Stock habis → → Payment Service: refund (compensation)
6. ✓ Stock cukup → → Shipping Service: schedule shipping
7. → Selesai

```
// order-service/saga/orchestrator.ts
```

```
enum SagaStep {
  CREATE_ORDER = 'CREATE_ORDER',
  PROCESS_PAYMENT = 'PROCESS_PAYMENT',
  REDUCE_STOCK = 'REDUCE_STOCK',
  SCHEDULE_SHIPPING = 'SCHEDULE_SHIPPING',
}

interface SagaState {
  orderId: string;
  currentStep: SagaStep;
  status: 'running' | 'completed' | 'compensating';
  data: any;
}

class SagaOrchestrator {
  async execute(orderData: OrderData) {
    const saga: SagaState = {
      orderId: uuidv4(),
      currentStep: SagaStep.CREATE_ORDER,
      status: 'running',
      data: orderData,
    };

    try {
      await this.stepCreateOrder(saga);
      await this.stepProcessPayment(saga);
      await this.stepReduceStock(saga);
      await this.stepScheduleShipping(saga);

      saga.status = 'completed';
      console.log(`[Saga] Order ${saga.orderId} completed successfully`);
    } catch (error) {
      console.error(`[Saga] Order ${saga.orderId} failed, compensating...`);
      saga.status = 'compensating';
      await this.compensate(saga);
    }
  }

  private async stepCreateOrder(saga: SagaState) {
```

```

    const order = await orderClient.createOrder(saga.data);
    saga.data.orderId = order.id;
    saga.currentStep = SagaStep.PROCESS_PAYMENT;
  }

  private async stepProcessPayment(saga: SagaState) {
    await paymentClient.processPayment({
      orderId: saga.data.orderId,
      amount: saga.data.total,
    });
    saga.currentStep = SagaStep.REDUCE_STOCK;
  }

  private async compensate(saga: SagaState) {
    // Reverse steps in reverse order
    switch (saga.currentStep) {
      case SagaStep.SCHEDULE_SHIPPING:
        await shippingClient.cancelShipping({ orderId: saga.data.orderId });
      case SagaStep.REDUCE_STOCK:
        await inventoryClient.restoreStock({ orderId: saga.data.orderId });
      case SagaStep.PROCESS_PAYMENT:
        await paymentClient.refundPayment({ orderId: saga.data.orderId });
      case SagaStep.CREATE_ORDER:
        await orderClient.cancelOrder({ orderId: saga.data.orderId });
    }
  }
}

```

### 4.3 Choreography vs Orchestration

| Aspek            | Choreography               | Orchestration               |
|------------------|----------------------------|-----------------------------|
| Kompleksitas     | Low (no central service)   | High (orchestrator logic)   |
| Coupling         | Event coupling (loose)     | Command coupling (tight)    |
| Tracing          | Hard (event chain panjang) | Easy (single flow)          |
| Failure handling | Distributed (event-based)  | Centralized (orchestrator)  |
| Cocok untuk      | Simple flow, few services  | Complex flow, many services |

## 5. Latihan: REST + Message Queue

### Studi Kasus

Buat komunikasi antara **Order Service** dan **Inventory Service**:

**Order Service** (REST API): - POST /api/orders — checkout order -  
Panggil Inventory Service via REST untuk cek stock

**Inventory Service** (REST API + MQ): - GET /internal/stock/:productId  
— cek stock - Consume event order.created untuk update stock -  
Publish event stock.updated setelah update

### Tugas

1. **REST sync call**: Order Service cek stock ke Inventory Service via REST
2. **Async event**: Order Service publish order.created ke RabbitMQ
3. **Consumer**: Inventory Service consume event, update stock
4. **Failure handling**: Jika stock kurang → publish order.stock\_insufficient
5. **Idempotency**: Pastikan event ga diproses 2x

### Template Jawaban

```
// order-service/src/services/checkout.ts
async function checkout(userId: string, items: CartItem[]) {
  // 1. Cek stock via REST
  for (const item of items) {
    const stock = await inventoryClient.checkStock(item.productId);
    if (stock < item.quantity) throw new Error(`Insufficient stock for ${item.pr
  }

  // 2. Buat order
  const order = await Order.create({ userId, items, status: 'pending' });

  // 3. Publish event
  await messageBus.publish('orders.direct', 'order.created', {
    orderId: order.id,
    items,
  });

  return order;
}

// inventory-service/src/consumers/order-consumer.ts
// TODO: consume 'order.created', update stock, handle insufficient
```

---

## □ Referensi

- [gRPC Official Guide](#)
- [RabbitMQ Tutorials](#)
- [Saga Pattern — Microsoft](#)
- [Outbox Pattern](#)
- [Opossum Circuit Breaker](#)

## Sesi 04: API Gateway

**Durasi:** 120 menit

**Prerequisites:** Sesi 01-03

---

## □ Tujuan

- Memahami fungsi API Gateway (routing, aggregation, auth, rate limit)
  - Setup Traefik sebagai API Gateway
  - Implementasi service discovery
  - Circuit breaker di gateway level
  - Log aggregation & correlation ID
- 

## 1. API Gateway — Konsep

### 1.1 Tanpa vs Dengan Gateway

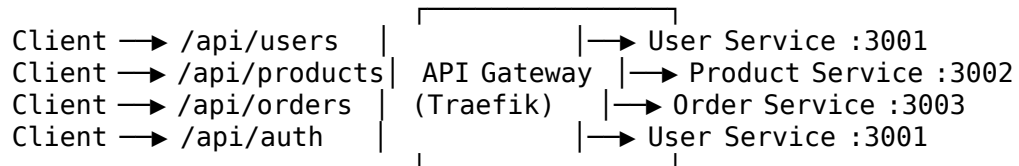
Tanpa API Gateway:

|        |                        |                         |
|--------|------------------------|-------------------------|
| Client | → POST /api/users      | → User Service :3001    |
| Client | → POST /api/products   | → Product Service :3002 |
| Client | → POST /api/orders     | → Order Service :3003   |
| Client | → POST /api/auth/login | → User Service :3001    |
| Client | → POST /api/payments   | → Order Service :3003   |

Masalah:

- Client tahu detail internal (port, host, protocol)
- Auth logic duplikasi di setiap service
- CORS, rate limit, logging harus di setiap service
- Hard to change: rename service? ganti port? semua client harus update

Dengan API Gateway:



Keuntungan:

- Client cuma tau satu endpoint (gateway)
- Auth centralized di gateway
- Rate limiting, logging, CORS satu tempat
- Service bisa ganti port/host tanpa client tahu

## 1.2 Fungsi Utama API Gateway

| Fungsi          | Deskripsi                               | Contoh                    |
|-----------------|-----------------------------------------|---------------------------|
| Routing         | Forward request ke service yang tepat   | /api/users → user-service |
| Aggregation     | Gabung response dari multiple service   | Dashboard: user + orders  |
| Auth            | Validasi token sebelum request masuk    | JWT verification          |
| Rate Limiting   | Batasi jumlah request per client        | 100 req/min per IP        |
| Load Balancing  | Distribute request ke multiple instance | Round-robin ke 3 instance |
| Circuit Breaker | Stop request ke service yang down       | Fallback response         |
| Logging         | Centralized request/response log        | Correlation ID            |

## 1.3 API Gateway Patterns

*// Pattern 1: Gateway Routing – simple proxy*

*// Client → Gateway → Service*

*// Gateway cuma forward, ga ubah request/response*

*// Pattern 2: Gateway Aggregation – composite response*

*// Client → Gateway → Service A + Service B → combine → Client*

*// Berguna untuk dashboard yang butuh data dari banyak service*

```
async function getUserDashboard(userId: string) {
```

```

const [user, orders, products] = await Promise.all([
  userService.getUser(userId),
  orderService.getOrders(userId),
  productService.getRecentProducts(),
]);

return {
  profile: user,
  recentOrders: orders.slice(0, 5),
  recommendedProducts: products.slice(0, 3),
};
}

// Pattern 3: Gateway Offloading – auth di gateway
// Gateway handle JWT validation, service terima user info sudah valid
// Service ga perlu tau soal token – trusted header dari gateway

```

## 2. Traefik API Gateway

### 2.1 Kenapa Traefik?

| Fitur             | Traefik                          | Nginx              | Express Gateway |
|-------------------|----------------------------------|--------------------|-----------------|
| Konfigurasi       | Dynamic (auto-detect containers) | Static file        | Static file     |
| Service discovery | Docker, Kubernetes, Consul       | Manual upstream    | Manual          |
| Hot reload        | Yes (no restart)                 | No (reload config) | No (restart)    |
| Dashboard GUI     | ☐                                | ☐                  | ☐               |
| Let's Encrypt     | Auto                             | Manual             | Manual          |
| Middleware        | Built-in                         | Lua/3rd party      | Plugin          |

### 2.2 Traefik Configuration

```

# gateway/traefik.yml – static configuration
api:
  dashboard: true

```

```

    debug: true

entryPoints:
  web:
    address: ":80"
  websecure:
    address: ":443"

providers:
  docker:
    endpoint: "unix:///var/run/docker.sock"
    exposedByDefault: false    # Hanya service dengan label traefik.enable=true
    swarmMode: false
    network: ms-backend

  file:
    filename: /etc/traefik/dynamic.yml
    watch: true

certificatesResolvers:
  letsencrypt:
    acme:
      email: admin@example.com
      storage: /etc/traefik/acme.json
      httpChallenge:
        entryPoint: web

# Global middleware
log:
  level: INFO
  filePath: /var/log/traefik.log

accessLog:
  filePath: /var/log/access.log
  format: json

# gateway/dynamic.yml – dynamic routing rules
http:
  routers:
    # =====
    # User Service Routes
    # =====
    user-service-api:
      rule: "PathPrefix(`/api/users`) || PathPrefix(`/api/auth`)"
      service: user-service
      entryPoints:

```

```

        - web
middlewares:
    - auth-middleware
    - rate-limit
    - cors-headers
priority: 10

user-service-health:
    rule: "PathPrefix(`/health/user`)"
    service: user-service
    entryPoints:
        - web
    middlewares: []
    # No auth untuk health check

# =====
# Product Service Routes
# =====
product-service-api:
    rule: "PathPrefix(`/api/products`)"
    service: product-service
    entryPoints:
        - web
    middlewares:
        - auth-middleware
        - rate-limit
        - cors-headers
    priority: 10

# =====
# Order Service Routes
# =====
order-service-api:
    rule: "PathPrefix(`/api/orders`)"
    service: order-service
    entryPoints:
        - web
    middlewares:
        - auth-middleware
        - rate-limit
        - cors-headers
    priority: 10

# =====
# Dashboard (Traefik)
# =====

```

```

dashboard:
  rule: "PathPrefix(`/api/dashboard`) || PathPrefix(`/dashboard`)"
  service: api@internal
  entryPoints:
    - web
  middlewares:
    - dashboard-auth
  priority: 1

# =====
# Middleware Definitions
# =====
middlewares:
  # Auth – validasi JWT
  auth-middleware:
    forwardAuth:
      address: "http://user-service:3001/internal/verify-token"
      trustForwardHeader: true
    authResponseHeaders:
      - X-User-Id
      - X-User-Role
      - X-User-Email

# Rate Limiting
rate-limit:
  rateLimit:
    average: 100          # 100 request per...
    period: 1m            # ...menit
    burst: 10             # burst allowance
    sourceCriterion:
      ipStrategy:
        depth: 1

# CORS
cors-headers:
  headers:
    accessControlAllowOriginList:
      - "http://localhost:3000"
      - "https://app.example.com"
    accessControlAllowMethods:
      - GET
      - POST
      - PUT
      - DELETE
      - PATCH
    accessControlAllowHeaders:

```

```

        - Authorization
        - Content-Type
        - X-Correlation-Id
    accessControlExposeHeaders:
        - X-Correlation-Id
        - X-Request-Id
    accessControlAllowCredentials: true

# Dashboard basic auth
dashboard-auth:
  basicAuth:
    users:
      - "admin:$2y$10$..." # bcrypt hash

# =====
# Services (Backends)
# =====
services:
  user-service:
    loadBalancer:
      servers:
        - url: "http://user-service:3001"
      healthCheck:
        path: /health
        interval: 30s
        timeout: 5s

  product-service:
    loadBalancer:
      servers:
        - url: "http://product-service:3002"
      healthCheck:
        path: /health
        interval: 30s
        timeout: 5s

  order-service:
    loadBalancer:
      servers:
        - url: "http://order-service:3003"
      healthCheck:
        path: /health
        interval: 30s
        timeout: 5s

```

## 2.3 Docker Compose Labels (Auto-Discovery)

Alternatif — tanpa dynamic.yml, bisa pakai Docker labels langsung:

```
# docker-compose.yml – Traefik auto-discover via labels
services:
  user-service:
    image: user-service:latest
    labels:
      - "traefik.enable=true"

      # Router
      - "traefik.http.routers.user-service.rule=PathPrefix(`/api/users`) || PathPrefix(`/api/users/`)"
      - "traefik.http.routers.user-service.entrypoints=web"
      - "traefik.http.routers.user-service.middlewares=auth-middleware,rate-limit"

      # Service
      - "traefik.http.services.user-service.loadbalancer.server.port=3001"

      # Middleware (global, shared via file provider)
      - "traefik.http.middlewares.rate-limit.ratelimit.average=100"
      - "traefik.http.middlewares.rate-limit.ratelimit.period=1m"

gateway:
  image: traefik:v3.0
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.dashboard.rule=PathPrefix(`/dashboard`)"
    - "traefik.http.routers.dashboard.service=api@internal"
```

---

## 3. Service Discovery

### 3.1 DNS-Based Discovery

Paling sederhana — Docker Compose built-in DNS.

```
// Setiap container bisa akses service lain via service name
const USER_SERVICE_URL = 'http://user-service:3001';
const PRODUCT_SERVICE_URL = 'http://product-service:3002';
const ORDER_SERVICE_URL = 'http://order-service:3003';

// Docker DNS otomatis resolve nama service ke IP container
// Tanpa perlu hardcode IP!
```

### 3.2 Consul Service Discovery

Untuk produksi — dynamic registration & health check.

**// Service registration – dijalankan saat service start**

```
{
  "ID": "user-service-1",
  "Name": "user-service",
  "Address": "10.0.1.5",
  "Port": 3001,
  "Tags": ["api", "v1"],
  "Check": {
    "HTTP": "http://10.0.1.5:3001/health",
    "Interval": "10s",
    "Timeout": "5s"
  }
}
```

*// consul-client.ts – dynamic service resolution*

```
import Consul from 'consul';
```

```
const consul = new Consul({ host: 'consul', port: 8500 });
```

```
async function discoverService(serviceName: string): Promise<string> {
  const services = await consul.agent.service.list();
```

```
  const instances = Object.values(services).filter(
    (s: any) => s.Service === serviceName && s.Checks.every((c: any) => c.Status === 'passing')
  );
```

```
  if (instances.length === 0) {
    throw new Error(`No healthy instances of ${serviceName}`);
  }
```

*// Load balancing: random instance*

```
  const instance = instances[Math.floor(Math.random() * instances.length)] as any;
  return `http://${instance.Address}:${instance.Port}`;
```

```
}
```

*// Usage – resolve setiap request*

```
async function callUserService(path: string) {
  const url = await discoverService('user-service');
  const response = await axios.get(`${url}${path}`);
  return response.data;
}
```

### 3.3 Kubernetes Service Discovery

```
# Kubernetes – built-in DNS
# Service dns: <service-name>.<namespace>.svc.cluster.local

# otomatis load balance ke semua pod
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  ports:
    - port: 80
      targetPort: 3001

// K8s – tinggal pake service name
const USER_SERVICE_URL = 'http://user-service.default.svc.cluster.local';
```

---

## 4. Circuit Breaker di Gateway

### 4.1 Traefik Circuit Breaker

```
# Traefik circuit breaker – di service definition
services:
  user-service:
    loadBalancer:
      servers:
        - url: "http://user-service:3001"
    healthCheck:
      path: /health
      interval: 5s
    # Circuit breaker
    serversTransport:
      # Timeout per request
      responseForwarding:
        flushInterval: 100ms
    sticky:
      cookie:
        name: user_service_sticky
```

## 4.2 Opossum — Application-Level Circuit Breaker

```
// gateway/src/middleware/circuit-breaker.ts
import CircuitBreaker from 'opossum';
import { Request, Response, NextFunction } from 'express';

const breakers = new Map<string, CircuitBreaker>();

function createServiceBreaker(serviceName: string, baseUrl: string) {
  const breaker = new CircuitBreaker(async (req: Request) => {
    const response = await axios({
      method: req.method,
      url: `${baseUrl}${req.path}`,
      data: req.body,
      headers: req.headers,
      timeout: 5000,
    });
    return response.data;
  }, {
    timeout: 3000,
    errorThresholdPercentage: 50,
    resetTimeout: 30000,
    name: serviceName,
  });

  // Fallback
  breaker.fallback(() => ({
    success: false,
    error: `${serviceName} is currently unavailable. Please try again later.`,
    errorCode: 'SERVICE_UNAVAILABLE',
  }));

  // Log state changes
  breaker.on('open', () => console.warn(`[CB] ${serviceName} circuit OPEN`));
  breaker.on('halfOpen', () => console.info(`[CB] ${serviceName} circuit HALF-OPEN`));
  breaker.on('close', () => console.info(`[CB] ${serviceName} circuit CLOSED`));

  return breaker;
}

// Middleware
function circuitBreakerMiddleware(services: Record<string, string>) {
  return (req: Request, res: Response, next: NextFunction) => {
    const serviceName = extractServiceName(req.path);
    const baseUrl = services[serviceName];
  }
}
```

```

    if (!baseUrl) return next(); // Pass through if unknown

    if (!breakers.has(serviceName)) {
        breakers.set(serviceName, createServiceBreaker(serviceName, baseUrl));
    }

    const breaker = breakers.get(serviceName)!;

    breaker.fire(req)
        .then((data) => res.json(data))
        .catch((err) => {
            res.status(503).json({
                success: false,
                error: `Service ${serviceName} unavailable`,
                circuitStatus: breaker.status,
            });
        });
    };
}

```

---

## 5. Log Aggregation & Correlation ID

### 5.1 Centralized Logging

```

// shared/logger.ts – structured logging dengan correlation ID
import winston from 'winston';
import { AsyncLocalStorage } from 'async_hooks';

const asyncStorage = new AsyncLocalStorage<{ correlationId: string; service: string }>();

const logger = winston.createLogger({
    format: winston.format.combine(
        winston.format.timestamp(),
        winston.format.errors({ stack: true }),
        winston.format.json(),
    ),
    defaultMeta: {
        service: process.env.SERVICE_NAME || 'unknown',
        environment: process.env.NODE_ENV || 'development',
    },
    transports: [
        new winston.transports.Console({
            format: winston.format.combine(
                winston.format.colorize(),
            ),
        })
    ]
});

```

```

        winston.format.printf(({ timestamp, level, message, correlationId, service }) => {
            const cid = correlationId || asyncStorage.getStore()?.correlationId ||
            return `${timestamp} [${service}] [${cid}] ${level}: ${message} ${
                Object.keys(meta).length ? JSON.stringify(meta) : ''
            }`;
        }),
    ),
    },
    new winston.transports.File({
        filename: `/var/log/app/${process.env.SERVICE_NAME}.log`,
        maxsize: 10 * 1024 * 1024, // 10MB
        maxFiles: 5,
    }),
],
});

// Wrapper dengan async context
export function log(level: string, message: string, meta: Record<string, any> = {}) {
    const store = asyncStorage.getStore();
    logger.log(level, message, {
        ...meta,
        correlationId: store?.correlationId,
    });
}

export const logInfo = (msg: string, meta?: any) => log('info', msg, meta);
export const logError = (msg: string, meta?: any) => log('error', msg, meta);
export const logWarn = (msg: string, meta?: any) => log('warn', msg, meta);

```

## 5.2 Correlation ID Propagation

```

// gateway/src/middleware/correlation.ts
import { v4 as uuidv4 } from 'uuid';

function correlationMiddleware(req: Request, res: Response, next: NextFunction) {
    // Ambil dari header incoming, atau buat baru
    const correlationId = (req.headers['x-correlation-id'] as string) || uuidv4();

    // Simpan di async context
    const store = { correlationId, service: 'api-gateway' };

    asyncStorage.run(store, () => {
        // Set response header
        res.setHeader('X-Correlation-Id', correlationId);
    });
}

```

```

    // Attach ke outgoing request (axios interceptor)
    req.correlationId = correlationId;
    req.outgoingHeaders = {
      'X-Correlation-Id': correlationId,
    };

    next();
  });
}

// Axios global interceptor – auto attach correlation ID
axios.interceptors.request.use((config) => {
  const store = asyncStorage.getStore();
  if (store?.correlationId) {
    config.headers['X-Correlation-Id'] = store.correlationId;
  }
  return config;
});

```

### 5.3 ELK Stack Integration

```

# docker-compose – logging infrastructure
services:
  # Filebeat – collect logs from all services
  filebeat:
    image: docker.elastic.co/beats/filebeat:8.12.0
    volumes:
      - /var/lib/docker/containers:/var/lib/docker/containers:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./gateway/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
    depends_on:
      - elasticsearch

# Elasticsearch – store & index logs
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.12.0
  environment:
    - discovery.type=single-node
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
  volumes:
    - elasticsearch-data:/usr/share/elasticsearch/data
  ports:
    - "9200:9200"

# Kibana – visualize logs

```

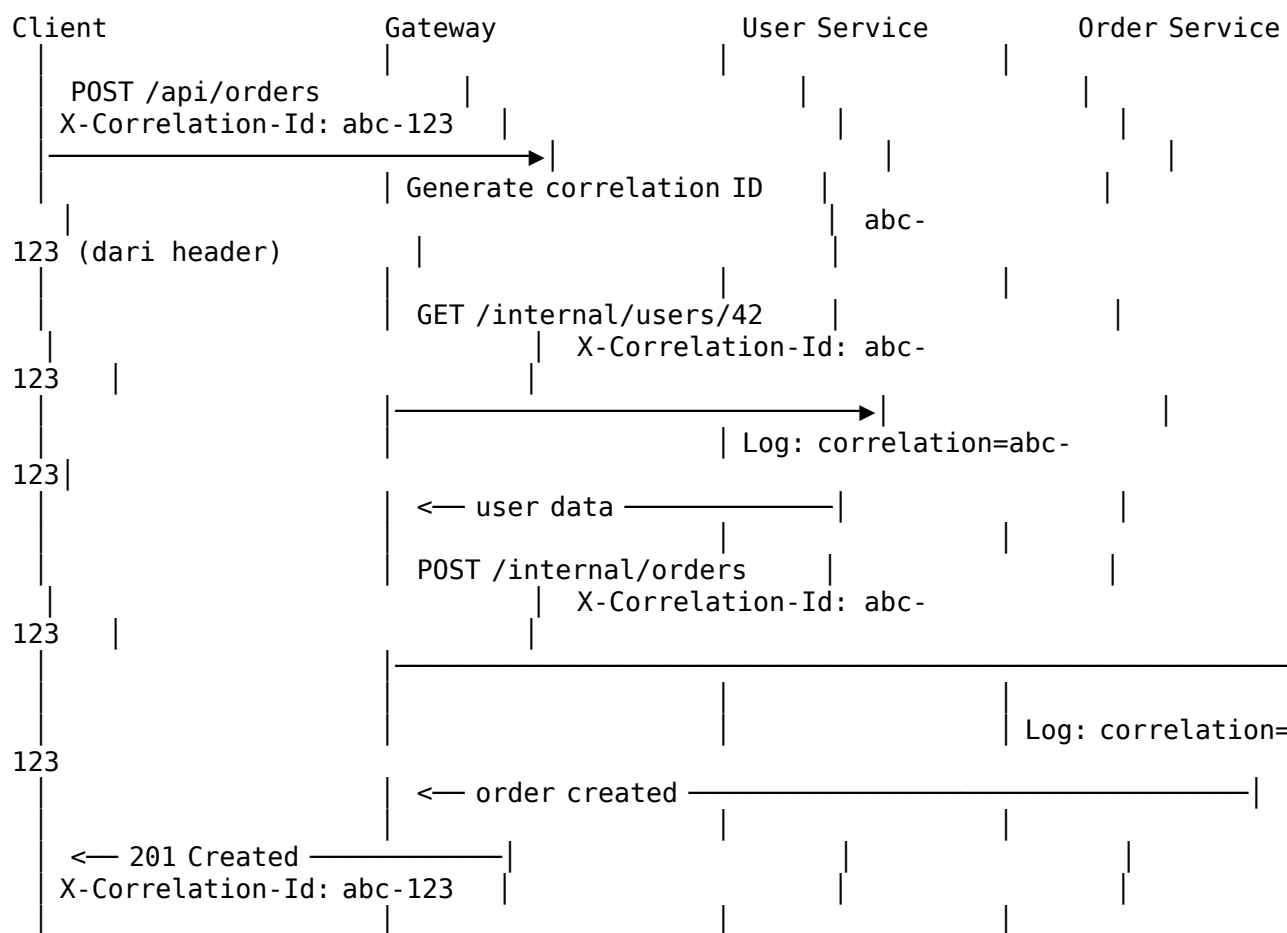
```

kibana:
  image: docker.elastic.co/kibana/kibana:8.12.0
  ports:
    - "5601:5601"
  depends_on:
    - elasticsearch

```

## 5.4 Tracing Correlation ID Across Services

Request flow dengan correlation ID:



**Debugging:** Cari semua log dengan correlationId=abc-123 → lihat seluruh perjalanan request.

## 6. Latihan: Setup Traefik API Gateway

### Studi Kasus

Setup Traefik sebagai API Gateway untuk microservices yang sudah ada (User, Product, Order Service).

### Spesifikasi

1. **Routing Rules:**
  - /api/users/\* → user-service:3001
  - /api/products/\* → product-service:3002
  - /api/orders/\* → order-service:3003
2. **Middleware:**
  - Auth: forward auth ke user-service /internal/verify-token
  - Rate limit: 50 request/menit per IP
  - CORS: izinkan localhost:3000
3. **Health Check:**
  - Setiap service punya health check endpoint
  - Traefik detect service health auto
4. **Dashboard:**
  - Traefik dashboard di /dashboard
  - Protected dengan basic auth

### Tugas

1. Buat gateway/traefik.yml (static config)
2. Buat gateway/dynamic.yml (routing rules)
3. Update docker-compose.yml — tambah service gateway + label
4. Test routing: curl localhost/api/users/health → response dari user-service

### Template Jawaban

```
# gateway/traefik.yml
# TODO: static configuration

# gateway/dynamic.yml
# TODO: routers, middlewares, services
```

---

### □ Referensi

- [Traefik Documentation](#)

- [API Gateway Pattern — AWS](#)
- [Opossum Circuit Breaker](#)
- [Correlation ID Pattern](#)
- [ELK Stack — Elastic](#)